

AI-Driven Portfolio Protection

Simple-Modelling Track — Chapters 4 and 5 (working draft)

Fiyinfo luwa Akano

URN 6962514

Supervisor: Dr Cuong Nguyen

August 4, 2026

Chapter 4

System Architecture & Implementation

This chapter describes how the mathematical model of Chapter 3 becomes running Python code. The implementation framework translates the Markov Decision Process (MDP) established in Chapter 3 into a modular execution system built on Gymnasium and PyTorch, separating market state handling, policy network evaluation, transaction verification, and performance evaluation. Everything in this chapter is verifiable in the public repository (<https://github.com/TheFinix13/dissertation-project>, branch `simple-modelling-iteration1`).

4.1 Overview of system design

Figure 4.1 shows the single loop everything else plugs into. The policy network $\pi_\theta(a | s)$ proposes a discrete action a_t ; the trading environment (`TradingEnv`, a `Gymnasium gym.Env` subclass) executes the action against the market data stream, then returns the next state s_{t+1} and the reward R_t . Training is nothing more than running this cycle many times and adjusting θ so actions that increased wealth become more probable.

4.2 Environment engine (`TradingEnv`)

The core environment subclasses `gym.Env` and handles market data loading, position state tracking, order execution checks, and state vector formulation. To handle experimental progression alongside Chapter 3, the codebase scales incrementally from the minimal baseline to further iterations.

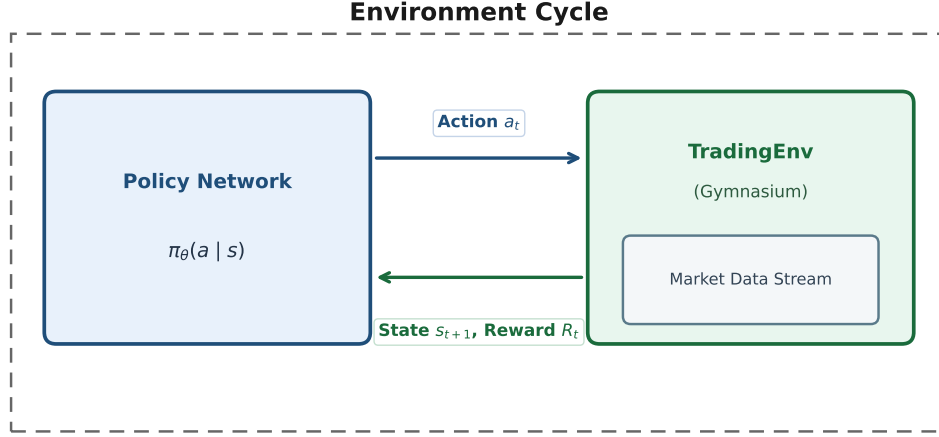


Figure 4.1: The environment cycle. The policy network exchanges actions for states and rewards with the Gymnasium trading environment, which wraps the market data stream.

4.2.1 Iteration 1 baseline engine (TradingEnv-v0)

The first engine (`experiments/iteration1/env.py`) works on a 3-dimensional state comprising the one-step price return, total liquid cash, and total shares held:

$$s_t = [\Delta P_t, C_t, n_t]^\top. \quad (4.1)$$

The action set is $\mathcal{A} = \{0 : \text{Hold}, 1 : \text{Buy one share}, 2 : \text{Sell one share}\}$ with a feasibility mask (Section 4.3). Wealth is marked to market as $W_t = C_t + n_t P_t$ and the per-step reward is the wealth change $r_t = W_{t+1} - W_t$.

4.2.2 Iteration 2 extended engine (TradingEnv-v1)

The 3-D state gives the agent its balance sheet and one-minute momentum, but leaves three gaps. **Gap A (position value)**: the network sees C_t and n_t but neural networks are poor at multiplying inputs on the fly, so it cannot judge the dollar exposure $n_t P_t$ against its cash. **Gap B (entry context)**: once holding, the agent has amnesia about its purchase price, so it cannot distinguish locking in a profit from cutting a loss. **Gap C (time)**: without knowing how far the episode has progressed, a Buy at the first bar looks identical to a Buy at the last bar.

TradingEnv-v1 (`experiments/iteration1/env_v1.py`) closes the gaps with a scaled 5-dimensional state:

$$s_t = \left[\Delta P_t, \text{PnL}_t, \tau_t, \frac{C_t}{C_0}, \frac{n_t P_t}{C_0} \right]^\top, \quad (4.2)$$

where PnL_t is the unrealized return of the open position against its average entry price (zero when flat), $\tau_t = t/T \in [0, 1]$ is episode progress, and the last two entries express

cash and position market value as fractions of starting capital C_0 so all features live on comparable scales. Actions, fees, masking and the reward are byte-identical to Iteration 1, so any performance difference between the two engines is attributable to the state representation alone.

4.3 State transition mechanics

When the agent issues action a_t , the environment performs the following checked transitions, with transaction cost rate $c = 0.0005$ (5 basis points):

- **Buy** ($a_t=1$): valid if $C_t \geq P_t(1 + c)$; then $C_{t+1} = C_t - P_t(1 + c)$ and $n_{t+1} = n_t + 1$.
- **Sell** ($a_t=2$): valid if $n_t \geq 1$; then $C_{t+1} = C_t + P_t(1 - c)$ and $n_{t+1} = n_t - 1$. When n returns to zero the entry-cost accumulator resets.
- **Hold** ($a_t=0$): balances unchanged.

Infeasible requests are exposed to the agent through an action mask and, if requested anyway, are executed as Hold. The rate at which the trained policy requests infeasible actions is reported in Chapter 5 as a constraint-awareness diagnostic.

4.4 Algorithm implementations

Five algorithms are implemented, deliberately spanning the value-based and policy-based families so that the choice of policy-gradient methods for trading is evidenced rather than asserted.

1. **Tabular Q-learning** (`run_cartpole_qlearning.py`). The Bellman update $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ applied to a lookup table. It requires hand-binning the continuous state ($6 \times 6 \times 12 \times 12$ bins for CartPole), which is precisely why the table cannot scale to trading states.
2. **DQN** (Stable-Baselines3). Replaces the table with a network Q_ϕ trained by mean-squared error to the Bellman target, with experience replay and a frozen target network.
3. **REINFORCE** — implemented *from scratch* in PyTorch (`experiments/phase0_games/reinforce.py`) not through a library, so the update is fully auditable: the per-episode loss is $L(\theta) = -\sum_t \log \pi_\theta(a_t | s_t) G_t$ with discounted return G_t normalised for variance reduction.
4. **A2C** (Stable-Baselines3). Actor-critic: the critic V_ϕ replaces the Monte-Carlo return with an advantage estimate, trading REINFORCE's variance for bias.

5. **PPO** (Stable-Baselines3). Clipped policy gradient, $\min(\rho_t A_t, \text{clip}(\rho_t, 1 \pm \epsilon) A_t)$ with $\epsilon = 0.2$, preventing any single update from destroying the policy. PPO is the algorithm carried forward into the trading experiments.

4.5 Phase 0: the game correctness sandbox

A trading agent that loses money is ambiguous: the market may be unlearnable, or the code may be wrong. A game removes the ambiguity because the correct outcome is known. Phase 0 therefore validates every algorithm implementation on three Gymnasium games of increasing difficulty before any market data is touched:

- **CartPole-v1**: 4-D continuous state, 2 discrete actions, +1 per step upright (cap 500, solved ≈ 475). The basic sanity check.
- **FlappyBird-v0**: 12-feature state, 2 actions, sparse pipe rewards — a harder exploration problem.
- **LunarLander-v3**: 8-D state, 4 discrete engine actions, shaped landing reward (solved ≈ 200) — the closest game analogue to managing a position with several discrete trade actions.

Each suite (`experiments/phase0_games/`) runs Random, REINFORCE, A2C, PPO and DQN under one fixed seed and evaluation protocol (30 greedy episodes), writing per-episode learning curves and mean evaluation returns to JSON. Results appear in Chapter 5.

4.6 Data pipeline and episode construction

Market data is SPY daily adjusted close, 2018–2024, fetched once by `experiments/iteration1/fetch` and frozen to disk. One episode is one calendar month of daily closes ($T \approx 18$ –23 steps). This is stated honestly: it is not the 390-minute intraday session of Chapter 3’s idealised episode; the environment API is horizon-agnostic, so minute bars are a drop-in upgrade. Months are split chronologically — the first 58 months train, the final 26 months (November 2022 – December 2024) test — and are never shuffled, so no future information leaks into training.

4.7 Testing and reproducibility

Three properties are enforced by unit tests (`experiments/iteration1/test_env.py`) on both engines: (i) the *accounting identity* — summed per-step rewards must equal

the total wealth change of the episode to within 10^{-6} ; (ii) *mask correctness* — Buy is masked without sufficient cash, Sell is masked without shares, and an illegal request executes as Hold; (iii) *feature correctness* — the v1 unrealized-PnL and episode-progress features take their analytically expected values on a hand-constructed price path. Every training run pins its seed, budget, fee and data split inside the emitted results JSON, and every chart is regenerated deterministically from those JSONs by scripts under `reports/builders/`.

Chapter 5

Empirical Results & Analysis

This chapter reports every experiment run to date on the simple-modelling track, in the order the methodology builds: Phase 0 validates the learning algorithms on games with known outcomes; Phase 1 applies the validated PPO loop to real SPY data under Iteration 1 (3-D state) and Iteration 2 (5-D state). All numbers below are produced by committed scripts and are reproducible from the repository; none are illustrative placeholders.

5.1 Experimental setup and evaluation metrics

Phase 0 (games). Each algorithm trains under a fixed seed (42) with a fixed interaction budget (50,000 environment timesteps on CartPole; 150,000 on Flappy Bird and LunarLander; REINFORCE uses an equivalent episode budget), then is evaluated greedily over 30 fresh episodes. The metric is mean evaluation return.

Phase 1 (trading). SPY daily closes 2018–2024, split chronologically into 58 training months and 26 held-out test months (Nov 2022 – Dec 2024). Starting cash $C_0 = \$10,000$, transaction cost $c = 5$ basis points, PPO budget 80,000 timesteps, seed 42. Metrics per test month: wealth change ΔW (dollars), trades per month, and the win rate against the strongest passive baseline. Two diagnostics accompany every run: the accounting identity (rewards must sum to the wealth change; tolerance 10^{-6}) and the illegal-action request rate.

5.2 Phase 0: algorithm validation on games

Table 5.1 reports mean evaluation return for all five algorithms on the three games. Figure 5.1 shows the CartPole learning curves.

Table 5.1: Phase 0 mean evaluation return (30 greedy episodes, seed 42). CartPole is solved at ≈ 475 ; LunarLander at ≈ 200 . Tabular Q-learning is only defined on the hand-discretized CartPole.

Algorithm	CartPole-v1	FlappyBird-v0	LunarLander-v3
Random	28.8	-7.4	-183.4
Tabular Q-learning	500.0	—	—
DQN	500.0	7.0	177.6
REINFORCE (scratch)	292.2	7.1	15.3
A2C	500.0	4.6	-41.2
PPO	500.0	12.6	176.4

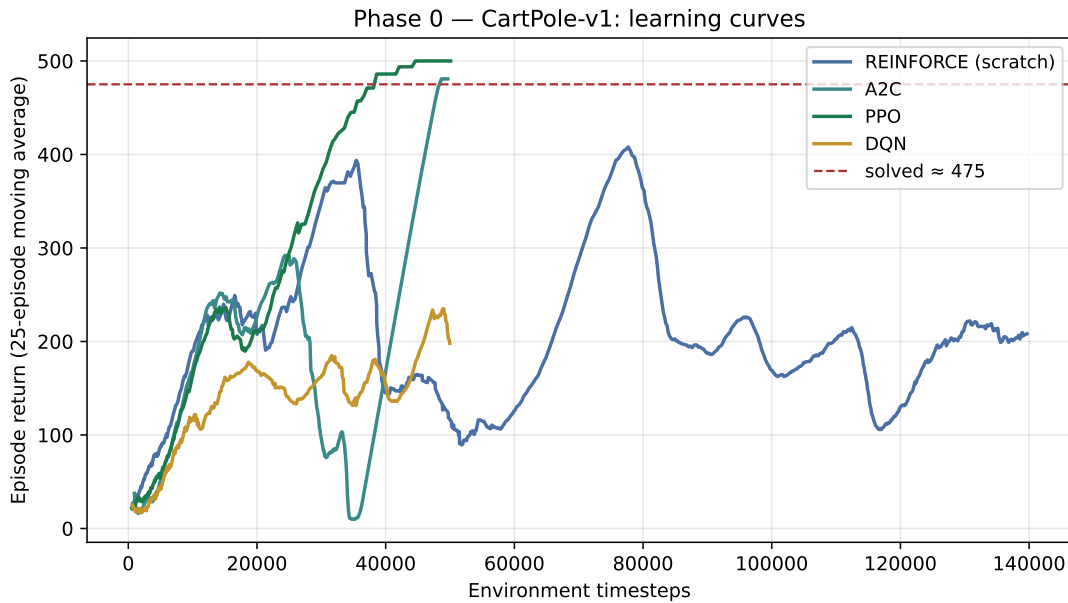


Figure 5.1: CartPole learning curves (25-episode moving average). All learning algorithms clear the random floor; A2C, PPO and DQN reach the 500-step cap within the 50k budget.

Three observations carry forward. First, every implementation clears its random floor on every game, which is the correctness evidence Phase 0 exists to provide. Second, the from-scratch REINFORCE learns on all three games but with visibly higher variance and a lower plateau — the textbook motivation for actor-critic and clipped updates. Third, tabular Q-learning reaches 500 on CartPole *only after* the continuous state is hand-binned into a $6 \times 6 \times 12 \times 12$ table; no such binning is available for a trading state with continuous cash and prices, which is why the trading experiments use policy-gradient methods with function approximation. On the harder games, PPO is the most consistent learner (best on Flappy Bird; within noise of DQN on LunarLander), supporting its selection for Phase 1.

5.3 Phase 1, Iteration 1: the 3-D state agent

Table 5.2 compares PPO on TradingEnv-v0 against four baselines with explicit objectives: B0 never trades; B1 buys a single share on the first bar and holds; B1b invests all available cash on the first bars and holds (the fair passive benchmark when $C_0 \gg P_t$); B2 selects uniformly among feasible actions.

Table 5.2: Iteration 1 on the 26 held-out SPY test months. ΔW in dollars; fee 5 bps; PPO budget 80k steps.

Method	Mean ΔW	Std	Trades/mo	Win vs B1b
B0 do-nothing	0.00	0.00	0.0	31%
B1 buy-one-hold	7.97	16.67	1.0	31%
B1b buy-max-hold	76.91	178.68	19.1	—
B2 random (masked)	14.14	32.93	12.8	31%
A1 PPO (3-D state)	76.91	178.68	19.1	0%

The honest headline. The trained PPO policy produced an action sequence *identical to B1b on all 26 test months*: under the pure wealth-change reward with a 3-D state, the optimal learned behaviour is simply “get fully invested and stay there.” The agent’s apparent +\$76.91 is therefore not evidence of skill beyond buy-and-hold — and this equivalence would have been invisible had the comparison stopped at the one-share baseline B1, which PPO “beats” by construction. The accounting identity held exactly on every episode, and the deterministic policy requested an infeasible action on 3.9% of steps (executed as Hold), indicating the constraints are mostly, but not perfectly, internalised.

5.4 Phase 1, Iteration 2: the 5-D state agent

Iteration 2 re-trains the identical PPO configuration on TradingEnv-v1, changing only the state vector (unrealized PnL, episode progress, and scaled balance-sheet features). Table 5.3 reports the comparison.

Table 5.3: Iteration 2 versus Iteration 1 on the same 26 test months, same budget, fee and seed.

Method	Mean ΔW	Trades/mo	Months identical to B1b
A1 PPO, 3-D state (Iter. 1)	76.91	19.1	26 / 26
A2 PPO, 5-D state (Iter. 2)	0.00	0.0	0 / 26
B1b buy-max-hold	76.91	19.1	—

The richer state *did* break the buy-and-hold collapse — the Iteration-2 policy matches B1b on zero months — but it converged to the opposite corner: a cautious, mostly-flat policy. Inspection of the policy’s action distribution over a representative test month shows a mean Hold probability of 0.66 (Buy 0.12, Sell 0.22), so the deterministic (argmax) evaluation never trades; sampling actions stochastically yields a small positive mean ΔW of +\$1.87. On month-by-month wealth the flat agent still beats the fully-invested benchmark in 8 of 26 months — the months when SPY fell.

5.5 Analysis and key findings

1. The reward, not the state, sets the incentive. Under $r_t = \Delta W_t$ in a mostly-rising test window, full investment is near-optimal, and the 3-D agent found exactly that. Adding entry and time context (Iteration 2) did not create trading skill; it moved the policy along the exposure axis, from always-invested to almost-never-invested. Both extremes are rational responses to a risk-neutral reward estimated from noisy monthly episodes — the training window (2018–2022) contains the COVID crash and the 2022 bear market, where staying flat avoids large negative rewards.

2. Baseline design is a result in itself. The single most informative row in this chapter is B1b. Without the fair fully-invested benchmark, Iteration 1 would have been reported as “PPO beats buy-and-hold by 10×” (against B1); with it, the correct statement is “PPO rediscovered buy-and-hold.”

3. What Iteration 3 must change. Because the state upgrade alone shifted exposure rather than timing, the next iteration targets the objective: a reward of the form $r_t = \Delta W_t - \lambda D_t$ (penalising drawdown D_t) gives the agent a reason to hold *selectively* — invested in calm regimes, flat in falling ones — which neither extreme policy can satisfy. The masked-action rate (3.9% in Iteration 1) and the accounting identity (exact in every run) carry forward unchanged as standing diagnostics.

Scope. These are single-seed results at a fixed budget, on monthly episodes of daily bars. Multi-seed confidence intervals, minute-bar episodes ($T=390$), and the drawdown-penalised reward are the planned extensions, in that order.